ONEDRAFT

CAD-AI — Aria Powered

Identity, Security & Authorization System

Version 0.1

Status: Implementation Specification
Database: PostgreSQL
API: REST / JSON
API Version: `/api/v1`
Identifiers: UUID
Security Model: Identity + Organization + Project + Capability Authorization
Authentication: Bearer Token / Session-Compatible Identity Layer
Authorization: RBAC + Scoped Capabilities
Audit: Append-Only Security Events
AI Layer: Aria Powered

---

1. SECURITY DESIGN PRINCIPLE

Security is a foundational application boundary of OneDraft.

The security subsystem shall establish and enforce:

Identity

Who is making a request.

Authentication

Whether that identity has successfully authenticated.

Organization Membership

Which organization the identity belongs to.

Project Authorization

Which projects the identity may access.

Capability Authorization

Which operations the identity may perform.

Service Authorization

Which internal services may communicate with one another.

Artifact Authorization

Which project artifacts may be read, generated, modified, or released.

Auditability

What security-sensitive action occurred, when it occurred, and under which identity or service authority.

Security controls must be applied before privileged domain operations execute.

---

2. SECURITY ARCHITECTURE

The security architecture becomes:

```
CLIENT
    ↓
IDENTITY
    ↓
AUTHENTICATION
    ↓
TOKEN VALIDATION
    ↓
ORGANIZATION CONTEXT
    ↓
PROJECT CONTEXT
    ↓
CAPABILITY AUTHORIZATION
    ↓
API
    ↓
DOMAIN SERVICE
    ↓
JOB / WORKER / EVENT BUS
```

No external client bypasses the API authorization boundary.

Internal services shall authenticate independently rather than implicitly trusting network location.

---

3. SECURITY DOMAINS

The security subsystem initially consists of:

```
identity
security
sessions
credentials
authorization
service_identity
```

```
secrets
audit
```

The existing `identity` schema remains the authoritative location for human identity records.

Security-specific records are layered around that identity model.

---

4. UNIVERSAL SECURITY RULES

Security-sensitive records use:

```
UUID
TIMESTAMPTZ
```

All timestamps are UTC.

Authentication credentials must never be stored in plaintext.

Secrets must never be persisted in ordinary application tables.

Tokens must never be logged in plaintext.

Passwords, where supported, must be stored only as strong password hashes.

Security events are append-only.

Authorization decisions must fail closed.

Unknown identities receive no permissions.

Unknown projects receive no permissions.

Unknown organizations receive no permissions.

---

5. IDENTITY MODEL

The existing:

```
identity.users
```

table remains the canonical human identity record.

The security subsystem does not duplicate the user identity.

The relationship remains:

```
USER
 ↓
ORGANIZATION MEMBERSHIP
 ↓
```

```
PROJECT ACCESS
  ↓
CAPABILITY
```

---

6. USER SECURITY PROFILE

`identity.user_security_profiles`

Fields:

```
user_id UUID PRIMARY KEY
account_status VARCHAR(32) NOT NULL
authentication_status VARCHAR(32) NOT NULL
email_verified BOOLEAN NOT NULL DEFAULT FALSE
mfa_enabled BOOLEAN NOT NULL DEFAULT FALSE
last_authenticated_at TIMESTAMPTZ
last_failed_authentication_at TIMESTAMPTZ
failed_authentication_count INTEGER NOT NULL DEFAULT 0
locked_until TIMESTAMPTZ
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

The profile contains security state but not plaintext credentials.

---

7. AUTHENTICATION IDENTITIES

`identity.authentication_identities`

Fields:

```
id UUID PRIMARY KEY
user_id UUID NOT NULL
provider VARCHAR(64) NOT NULL
provider_subject VARCHAR(512)
identifier VARCHAR(320)
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_auth_identity_user
idx_auth_identity_provider
idx_auth_identity_status
```

Unique provider identities must not be duplicated.

---

8. PASSWORD CREDENTIALS

Where password authentication is enabled:

```
credentials.password_credentials
```

Fields:

```
id UUID PRIMARY KEY
user_id UUID NOT NULL
password_hash TEXT NOT NULL
algorithm VARCHAR(64) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
last_used_at TIMESTAMPTZ
```

The database stores only the password hash.

Plaintext passwords are never persisted.

---

9. AUTHENTICATION SESSIONS

```
sessions.user_sessions
```

Fields:

```
id UUID PRIMARY KEY
user_id UUID NOT NULL
session_hash VARCHAR(128) NOT NULL
created_at TIMESTAMPTZ NOT NULL
expires_at TIMESTAMPTZ NOT NULL
revoked_at TIMESTAMPTZ
last_seen_at TIMESTAMPTZ
ip_metadata JSONB
client_metadata JSONB
status VARCHAR(32) NOT NULL
```

The server stores a hashed session identifier rather than a reusable plaintext credential.

---

10. ACCESS TOKENS

Access tokens shall be short-lived.

Conceptually:

```
ACCESS TOKEN
    ↓
AUTHENTICATION
    ↓
```

```
AUTHORIZATION
```

Token claims should contain only information required for authorization and request processing.

Conceptual claims:

```
sub
iss
aud
iat
exp
jti
organization_context
scopes
```

Sensitive personal information should not be embedded unnecessarily.

---

11. TOKEN REVOCATION

```
security.token_revocations
```

Fields:

```
id UUID PRIMARY KEY
token_id VARCHAR(128) NOT NULL
user_id UUID
reason VARCHAR(128)
revoked_at TIMESTAMPTZ NOT NULL
expires_at TIMESTAMPTZ
```

Revocation supports:

```
logout
credential compromise
administrative suspension
security incident
account termination
```

---

12. MULTI-FACTOR AUTHENTICATION

The architecture shall support MFA without coupling the application to a single provider.

Supported factor categories may include:

```
TOTP
WEBAUTHN
SECURITY_KEY
IDENTITY_PROVIDER
```

MFA configuration shall be represented independently from the user record.

---

13. MFA FACTORS

`credentials.mfa_factors`

Fields:

```
id UUID PRIMARY KEY
user_id UUID NOT NULL
factor_type VARCHAR(64) NOT NULL
provider VARCHAR(64)
public_metadata JSONB
secret_reference TEXT
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
last_used_at TIMESTAMPTZ
```

Secrets are referenced through the secret-management subsystem rather than stored directly.

---

14. ORGANIZATION SECURITY

The existing:

```
identity.organizations
identity.organization_members
```

tables remain the tenancy boundary.

Every authenticated request that accesses project data must resolve:

```
USER
 ↓
ORGANIZATION
 ↓
PROJECT
```

A valid user identity alone does not grant project access.

---

15. PROJECT ACCESS

Project access shall be derived from organization membership and project-specific authorization.

Conceptually:

```
USER
 ↓
ORGANIZATION MEMBERSHIP
 ↓
```

```
PROJECT
  ↓
PROJECT ROLE
  ↓
CAPABILITY
```

---

16. PROJECT MEMBERS

`authorization.project_members`

Fields:

```
project_id UUID NOT NULL
user_id UUID NOT NULL
role VARCHAR(64) NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Primary key:

```
(project_id, user_id)
```

---

17. ROLES

`authorization.roles`

Fields:

```
id UUID PRIMARY KEY
name VARCHAR(128) NOT NULL
scope VARCHAR(32) NOT NULL
description TEXT
system_role BOOLEAN NOT NULL DEFAULT FALSE
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Initial scopes:

```
SYSTEM
ORGANIZATION
PROJECT
```

---

18. INITIAL PROJECT ROLES

Initial OneDraft project roles:

```
PROJECT_OWNER
PROJECT_ADMIN
DESIGNER
DRAFTER
REVIEWER
CLIENT
VIEWER
```

These roles provide authorization defaults.

They do not replace capability-level enforcement.

---

19. CAPABILITIES

```
authorization.capabilities
```

Fields:

```
id UUID PRIMARY KEY
name VARCHAR(128) NOT NULL UNIQUE
resource VARCHAR(128) NOT NULL
action VARCHAR(64) NOT NULL
description TEXT
created_at TIMESTAMPTZ NOT NULL
```

Examples:

```
project:read
project:write

model:read
model:write

aria:execute
aria:approve

drawing:read
drawing:generate

redline:read
redline:write

compliance:read
compliance:execute

validation:read
validation:execute

document:read
document:generate
```

```
acceptance:write

admin:organization
admin:project
```

---

20. ROLE CAPABILITIES

```
authorization.role_capabilities
```

Fields:

```
role_id UUID NOT NULL
capability_id UUID NOT NULL
created_at TIMESTAMPTZ NOT NULL
```

Primary key:

```
(role_id, capability_id)
```

This provides the initial RBAC mapping.

---

21. DIRECT PROJECT CAPABILITIES

Certain deployments may require explicit project-level grants.

```
authorization.project_capabilities
```

Fields:

```
project_id UUID NOT NULL
user_id UUID NOT NULL
capability_id UUID NOT NULL
effect VARCHAR(16) NOT NULL
created_at TIMESTAMPTZ NOT NULL
expires_at TIMESTAMPTZ
```

Effects:

```
ALLOW
DENY
```

Explicit denials override inherited permissions.

---

22. AUTHORIZATION DECISION MODEL

Every protected operation evaluates:

```
IDENTITY
+
ACCOUNT STATUS
+
ORGANIZATION MEMBERSHIP
+
PROJECT MEMBERSHIP
+
ROLE
+
CAPABILITY
+
RESOURCE
+
ACTION
+
OBJECT STATE
```

The result is:

```
ALLOW
```

or:

```
DENY
```

---

23. FAIL-CLOSED AUTHORIZATION

If authorization cannot be determined because of:

```
missing identity
missing membership
unknown capability
invalid project
invalid role
expired credential
authorization service failure
```

the operation shall return:

```
DENY
```

The system must never interpret an authorization failure as permission.

---

24. RESOURCE AUTHORIZATION

Authorization shall operate at the resource level.

Examples:

```
project
building
level
space
element
drawing
sheet
revision
redline
validation_run
document_package
```

A user authorized for a project may access project resources only through that project's security context.

---

25. ARIA AUTHORIZATION

Aria operates as a controlled application actor.

Aria does not receive unrestricted database authority.

The architecture remains:

```
USER
 ↓
ARIA REQUEST
 ↓
AUTHORIZATION
 ↓
COMMAND SERVICE
 ↓
DOMAIN VALIDATION
 ↓
MODEL TRANSACTION
```

Aria cannot bypass:

```
authorization
constraints
revision control
validation
audit
```

---

26. ARIA APPROVAL BOUNDARY

Certain operations may require explicit authorization beyond `aria:execute`.

Examples include:

```
major model changes
release of final documents
acceptance state changes
regulated workflow transitions
project ownership changes
administrative operations
```

Such operations require:

```
aria:approve
```

or an equivalent human authorization capability.

---

27. SERVICE IDENTITIES

Internal services shall have machine identities.

```
service_identity.services
```

Fields:

```
id UUID PRIMARY KEY
service_name VARCHAR(128) NOT NULL UNIQUE
environment VARCHAR(32) NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Examples:

```
api
worker
geometry-engine
compliance-engine
validation-engine
documentation-engine
artifact-service
event-bus
aria-orchestrator
```

---

28. SERVICE AUTHORIZATION

```
authorization.service_capabilities
```

Fields:

```
service_id UUID NOT NULL
capability_id UUID NOT NULL
created_at TIMESTAMPTZ NOT NULL
```

Primary key:

```
(service_id, capability_id)
```

Internal services receive only the capabilities required for their function.

---

29. SERVICE-TO-SERVICE AUTHENTICATION

Internal communication shall use authenticated service credentials.

Conceptually:

```
SERVICE A
 ↓
SERVICE CREDENTIAL
 ↓
AUTHENTICATION
 ↓
SERVICE AUTHORIZATION
 ↓
SERVICE B
```

Network reachability alone is not authorization.

---

30. SERVICE TOKEN MODEL

Service credentials should support:

```
service_id
audience
issued_at
expires_at
token_id
capabilities
```

Short-lived credentials are preferred.

Long-lived secrets should be rotated.

---

31. SECRET MANAGEMENT

Secrets shall not be stored in:

```
source code
Git repositories
database records
OpenAPI examples
logs
event payloads
application errors
```

Secret references may be stored where required.

Actual secret material belongs in a dedicated secret-management mechanism.

---

32. ENVIRONMENT SEPARATION

OneDraft environments shall be separated:

```
development
test
staging
production
```

Credentials and secrets must not be shared between environments.

Production credentials must never be used by development workers.

---

33. API SECURITY MIDDLEWARE

Every protected API request passes through:

```
request_id
authentication
token_validation
identity_resolution
organization_resolution
project_resolution
authorization
rate_control
domain_service
audit
```

The exact middleware ordering must ensure authorization occurs before protected domain execution.

---

34. API SECURITY HEADERS

The API gateway should establish appropriate security headers for supported clients.

Examples include:

```
Content-Security-Policy
Strict-Transport-Security
X-Content-Type-Options
Referrer-Policy
```

Browser-specific policies belong at the web gateway layer.

---

35. TRANSPORT SECURITY

Production API communication shall use:

```
HTTPS
```

Internal service communication should also use authenticated encrypted transport where infrastructure supports it.

Plain HTTP shall not be used for production credential transmission.

---

36. REQUEST CORRELATION

Every request receives:

```
request_id
```

The identifier propagates through:

```
API
 ↓
DOMAIN
 ↓
JOB
 ↓
WORKER
 ↓
EVENT
 ↓
ARTIFACT
```

This connects security events to the broader OneDraft execution history.

---

37. SECURITY AUDIT EVENTS

`audit.security_events`

Fields:

```
id UUID PRIMARY KEY
event_type VARCHAR(128) NOT NULL
user_id UUID
service_id UUID
organization_id UUID
project_id UUID
request_id UUID
resource_type VARCHAR(128)
resource_id UUID
result VARCHAR(32) NOT NULL
reason TEXT
metadata JSONB
timestamp TIMESTAMPTZ NOT NULL
```

The table is append-only.

---

38. SECURITY EVENT TYPES

Initial events:

```
AUTHENTICATION_SUCCESS
AUTHENTICATION_FAILURE
LOGOUT
SESSION_CREATED
SESSION_REVOKED
MFA_ENABLED
MFA_DISABLED
PASSWORD_CHANGED
ACCOUNT_LOCKED
ACCOUNT_UNLOCKED
TOKEN_REVOKED
AUTHORIZATION_GRANTED
AUTHORIZATION_DENIED
PROJECT_ACCESS_GRANTED
PROJECT_ACCESS_REVOKED
ROLE_ASSIGNED
ROLE_REMOVED
SERVICE_AUTHENTICATED
SECRET_ROTATED
SECURITY_POLICY_CHANGED
```

---

39. AUDIT DATA MINIMIZATION

Security logs shall contain enough information to establish:

```
who
what
when
where
which project
which resource
which request
which result
```

but shall not unnecessarily contain:

```
passwords
access tokens
private keys
secret values
authentication secrets
```

---

40. AUTHORIZATION AUDIT

A denied operation shall produce an auditable event.

Example:

```
AUTHORIZATION_DENIED
```

with:

```
user_id
project_id
resource_type
resource_id
requested_capability
request_id
timestamp
```

This allows security investigation without exposing credentials.

---

41. RATE LIMITING

The API shall support rate controls at multiple levels:

```
IP
user
organization
project
API capability
service
```

Different operations may have different limits.

Expensive operations such as:

```
ARIA execution
validation
drawing generation
document generation
```

should primarily be governed through the existing asynchronous job architecture rather than unrestricted synchronous requests.

---

42. ABUSE PROTECTION

Repeated authentication failures may trigger temporary controls.

Examples:

```
authentication throttling
temporary account lock
token revocation
session invalidation
security event generation
```

Controls must be reversible and auditable.

---

43. PROJECT DATA ISOLATION

Every project query must be scoped by:

```
organization_id
project_id
```

The application shall never rely solely on a client-supplied project identifier.

The server resolves and verifies project ownership/access before retrieving project data.

---

44. DATABASE SECURITY BOUNDARY

Application services connect to PostgreSQL using service-specific database credentials.

The application shall not expose PostgreSQL credentials to:

```
frontend
browser
mobile client
external integration
Aria prompt
```

---

45. DATABASE LEAST PRIVILEGE

Separate database users should eventually exist for:

```
migration
application
worker
read-only reporting
administration
```

Application services should receive only the database privileges required by their function.

---

46. AI DATA BOUNDARY

Project data supplied to Aria shall pass through an explicit application boundary.

Conceptually:

```
PROJECT MODEL
 ↓
AUTHORIZED DATA SELECTION
 ↓
ARIA CONTEXT BUILDER
 ↓
MODEL PROVIDER
```

Aria should not receive unrelated projects or unrestricted database contents.

---

47. AI PROVIDER ISOLATION

The external AI provider remains behind:

```
Aria Orchestration
 ↓
AI Provider Adapter
```

The provider does not become the OneDraft system of record.

The OneDraft project model remains authoritative.

---

48. EVENT SECURITY

Events published through the existing Event Bus shall carry sufficient security context to identify:

```
organization
project
actor
service
request
revision
```

but events must not expose credentials or secrets.

---

49. EVENT CONSUMER AUTHORIZATION

Event consumers must verify that they are authorized to process the event type.

A worker must not automatically gain permission to perform every operation merely because it can consume an event.

---

50. ARTIFACT SECURITY

The existing Artifact Storage subsystem remains responsible for physical artifact storage.

The security layer governs access.

The authorization model becomes:

```
PROJECT
  ↓
ARTIFACT
  ↓
ACCESS POLICY
  ↓
AUTHORIZED REQUEST
  ↓
SIGNED / CONTROLLED ACCESS
```

Artifact identifiers must not themselves constitute authorization.

---

51. DOCUMENT RELEASE SECURITY

Final document packages shall require appropriate authorization before release.

The release chain becomes:

```
MODEL
  ↓
REVISION
  ↓
CODE MANIFEST
```

```
 ↓
VALIDATION
 ↓
DOCUMENT PACKAGE
 ↓
ACCEPTANCE
 ↓
AUTHORIZED RELEASE
```

The security subsystem verifies the actor's capability at the release boundary.

---

52. CUSTOMER ACCEPTANCE SECURITY

Acceptance operations require authenticated project membership.

The acceptance record must establish:

```
customer_id
project_id
revision_id
model_version_id
document_package_id
timestamp
authorization context
```

Acceptance cannot be attributed to an unidentified actor.

---

53. PROFESSIONAL REVIEW SECURITY

Professional review operations remain under the existing revision architecture.

The security layer verifies:

```
reviewer identity
project membership
required capability
review scope
authorization
```

The system records the security context associated with the review.

---

54. ADMINISTRATIVE SECURITY

Administrative capabilities shall be separated from normal project capabilities.

Examples:

```
admin:organization
admin:project
admin:security
admin:billing
```

Administrative permissions must never be inferred from ordinary project access.

---

55. ORGANIZATION OWNER

The organization owner receives administrative authority through explicit role assignment.

The system must not infer ownership merely because a user created a project.

---

56. ROLE CHANGE TRANSACTION

Role assignment must execute atomically.

Conceptually:

```
BEGIN
 ↓
verify actor authorization
 ↓
verify target user
 ↓
verify organization/project
 ↓
assign role
 ↓
record security event
 ↓
COMMIT
```

Failure results in:

```
ROLLBACK
```

---

57. PROJECT ACCESS REVOCATION

When access is revoked:

```
project membership
 ↓
disabled
 ↓
active sessions evaluated
 ↓
```

```
tokens evaluated
 ↓
security event
```

Previously issued credentials must not continue granting unauthorized project access.

---

58. ACCOUNT SUSPENSION

A suspended account cannot initiate new protected operations.

The system may additionally revoke:

```
sessions
access tokens
refresh credentials
active project access
```

depending on security policy.

---

59. ACCOUNT DELETION

User deletion must preserve necessary historical provenance.

Where the legal and operational model requires retaining historical events, references should remain without exposing unnecessary personal information.

The system must distinguish:

```
account deactivation
```

from:

```
physical identity deletion
```

---

60. SECURITY POLICY

```
security.security_policies
```

Fields:

```
id UUID PRIMARY KEY
scope VARCHAR(64) NOT NULL
organization_id UUID
project_id UUID
policy_type VARCHAR(128) NOT NULL
configuration JSONB NOT NULL
```

```
version INTEGER NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Policies may control:

```
MFA requirements
session duration
token lifetime
role restrictions
document release
administrative access
```

---

61. POLICY VERSIONING

Security policies shall be versioned.

A change must record:

```
previous policy
new policy
actor
timestamp
reason
```

The active policy is explicitly identifiable.

---

62. SECURITY CONFIGURATION

Configuration affecting security shall not be silently changed.

Examples:

```
token lifetime
MFA requirement
session timeout
role capability
rate limit
release authorization
```

Changes generate security audit events.

---

63. API AUTHORIZATION ENDPOINT

Conceptually:

```
GET /api/v1/auth/me
```

Response:

```
{
  "data": {
    "user_id": "uuid",
    "organization_id": "uuid",
    "roles": [],
    "scopes": []
  }
}
```

This allows clients to determine their effective authenticated context.

---

64. LOGIN ENDPOINT

Conceptually:

```
POST /api/v1/auth/login
```

Request:

```
{
  "email": "user@example.com",
  "password": "..."
}
```

Response:

```
{
  "data": {
    "access_token": "...",
    "expires_at": "ISO-8601"
  }
}
```

Actual credential material must never appear in logs.

---

65. LOGOUT ENDPOINT

```
POST /api/v1/auth/logout
```

The server invalidates the applicable session or credential state.

A logout operation generates:

```
LOGOUT
```

in the security audit stream.

---

66. PROJECT AUTHORIZATION ENDPOINT

Conceptually:

```
GET /api/v1/projects/{project_id}/authorization
```

Response:

```
{
  "data": {
    "project_id": "uuid",
    "roles": [
      "DESIGNER"
    ],
    "capabilities": [
      "project:read",
      "project:write",
      "aria:execute",
      "drawing:generate"
    ]
  }
}
```

The response describes the caller's effective capabilities.

---

67. ROLE ASSIGNMENT ENDPOINT

```
POST /api/v1/projects/{project_id}/members
```

Request:

```
{
  "user_id": "uuid",
  "role": "REVIEWER"
}
```

The server verifies that the caller possesses the required administrative capability.

---

68. ACCESS REVOCATION ENDPOINT

```
DELETE /api/v1/projects/{project_id}/members/{user_id}
```

The operation requires:

```
admin:project
```

or equivalent authority.

The revocation is recorded in the security audit stream.

---

69. AUTHORIZATION SERVICE

The first application-level security service shall expose a common interface.

Conceptually:

```
authorize(
    actor,
    capability,
    resource,
    context
)
```

Result:

```
ALLOW
DENY
```

The service becomes the common authorization boundary for:

```
API
Aria
workers
event consumers
artifact access
document release
administration
```

---

70. SECURITY CONTEXT

Every authenticated request creates a security context:

```
SecurityContext
```

containing:

```
actor_id
actor_type
organization_id
project_id
roles
capabilities
request_id
authentication_method
```

The context propagates through the application execution chain.

---

71. ACTOR TYPES

The system recognizes:

```
USER
SERVICE
ARIA
SYSTEM
```

Each actor type remains subject to authorization.

---

72. SYSTEM ACTOR

Certain automated operations require a system actor.

Examples:

```
scheduled cleanup
artifact lifecycle
event replay
system maintenance
```

System actions must still be explicitly authorized and audited.

---

73. ARIA ACTOR

Aria is represented as an application actor rather than as a human user.

The audit record therefore distinguishes:

```
requested_by = USER
executed_by = ARIA
```

where appropriate.

This preserves the distinction between user intent and automated execution.

---

74. COMMAND PROVENANCE

A model command shall preserve:

```
requesting_user
executing_actor
project_id
revision_id
command_id
request_id
timestamp
```

This connects authorization to the existing command and revision architecture.

---

75. SECURITY AND REVISION CONTROL

A revision cannot be created merely because a caller possesses a valid token.

The command must pass:

```
authentication
authorization
domain validation
constraint validation
revision validation
```

before the existing Project Model transaction commits.

---

76. SECURITY AND COMPLIANCE

The security subsystem does not replace the Compliance & Regulatory Engine.

Instead:

```
SECURITY
  ↓
AUTHORIZES ACTOR

COMPLIANCE
  ↓
VALIDATES PROJECT

DOMAIN
  ↓
APPLIES CHANGE
```

The two systems remain independent but coordinated.

---

77. SECURITY AND VALIDATION

Security authorization occurs before the requested operation.

Validation determines whether the resulting project state is acceptable.

Therefore:

```
AUTHORIZATION ≠ VALIDATION
```

Both are required.

---

78. SECURITY AND DOCUMENTATION

Documentation generation requires:

```
authorized request
+
valid model state
+
appropriate revision
```

The Documentation Engine remains responsible for generating drawings.

The security layer determines whether the caller may initiate or retrieve the operation.

---

79. SECURITY AND JOB EXECUTION

The existing Job & Worker Execution System remains responsible for asynchronous execution.

Every job shall retain:

```
requested_by
authorized_capabilities
organization_id
project_id
request_id
```

Workers revalidate authorization where required for privileged operations.

---

80. SECURITY AND EVENT BUS

The existing Event Bus remains responsible for event transport.

Security establishes:

```
who published
who may consume
what project context exists
what operation is authorized
```

Event transport itself does not become the authorization system.

---

81. SECURITY AND ARTIFACT STORAGE

Artifact storage remains responsible for:

```
blob persistence
hashes
versions
retention
retrieval
```

Security remains responsible for:

```
who may retrieve
who may create
who may release
who may delete
```

---

82. SECURITY TRANSACTION BOUNDARY

Security-sensitive mutations shall execute atomically with their corresponding authorization records where practical.

For example:

```
BEGIN
  ↓
verify authorization
  ↓
modify membership
  ↓
record security event
  ↓
COMMIT
```

A role change must not succeed without its corresponding security record.

---

83. SECURITY FAILURE MODEL

Security failures shall use standardized errors.

Example:

```
{
  "error": {
```

```
    "code": "AUTHORIZATION_DENIED",
    "message": "The authenticated actor does not possess the required capability.",
    "resolution": "Request the required project authorization."
  },
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

The response must not disclose sensitive authorization internals unnecessarily.

---

84. SECURITY ERROR CATEGORIES

Initial codes:

```
AUTHENTICATION_REQUIRED
AUTHENTICATION_FAILED
TOKEN_EXPIRED
TOKEN_REVOKED
ACCOUNT_SUSPENDED
ACCOUNT_LOCKED
MFA_REQUIRED
AUTHORIZATION_DENIED
PROJECT_ACCESS_DENIED
ORGANIZATION_ACCESS_DENIED
CAPABILITY_DENIED
SERVICE_AUTHENTICATION_FAILED
SERVICE_AUTHORIZATION_DENIED
SECURITY_POLICY_VIOLATION
```

---

85. SECURITY TESTING

The security implementation shall include tests for:

```
valid authentication
invalid authentication
expired token
revoked token
suspended account
missing organization membership
missing project membership
invalid role
missing capability
explicit denial
cross-project access
cross-organization access
unauthorized Aria command
unauthorized document release
unauthorized artifact retrieval
service authentication
```

```
service authorization
audit generation
```

---

86. CROSS-TENANT TEST

The system must explicitly test:

```
USER A
 ↓
ORGANIZATION A
 ↓
PROJECT A
```

attempting to access:

```
PROJECT B
```

The expected result is:

```
DENIED
```

and:

```
AUTHORIZATION_DENIED
```

must be recorded.

---

87. ARIA SECURITY TEST

The first Aria security integration test shall verify:

```
USER
 ↓
ARIA REQUEST
 ↓
AUTHORIZED COMMAND
 ↓
PROJECT MODEL
 ↓
REVISION
 ↓
AUDIT
```

and separately:

```
USER WITHOUT aria:execute
 ↓
ARIA REQUEST
 ↓
DENIED
```

```
 ↓
NO MODEL MUTATION
 ↓
SECURITY EVENT
```

No unauthorized command may produce project state.

---

88. WORKER SECURITY TEST

A worker attempting an unauthorized operation shall receive:

```
SERVICE_AUTHORIZATION_DENIED
```

and no model mutation shall occur.

---

89. DOCUMENT RELEASE TEST

The system shall verify:

```
authorized user
 ↓
valid project
 ↓
valid revision
 ↓
validated document package
 ↓
authorized release
```

An unauthorized user attempting the same operation must receive:

```
DOCUMENT_RELEASE_DENIED
```

---

90. SECURITY MIGRATION ORDER

The initial security migrations should follow:

```
026_user_security_profiles
027_authentication_identities
028_password_credentials
029_sessions
030_token_revocations
031_mfa
032_roles
033_capabilities
034_role_capabilities
035_project_members
```

```
036_project_capabilities
037_service_identities
038_service_capabilities
039_security_policies
040_security_events
041_security_indexes
042_security_seed_data
```

These migrations follow the already-established OneDraft database migration sequence.

---

91. SECURITY SEED DATA

Development environments should contain:

```
PROJECT_OWNER
PROJECT_ADMIN
DESIGNER
DRAFTER
REVIEWER
CLIENT
VIEWER
```

and the initial capabilities defined by the OneDraft API contract.

Development service identities should include:

```
api
worker
geometry-engine
compliance-engine
validation-engine
documentation-engine
artifact-service
event-bus
aria-orchestrator
```

All development credentials remain development-only.

---

92. IMPLEMENTATION ARTIFACTS

The implementation package shall consist initially of:

```
026_security_schema.sql
security_types.py
authentication_service.py
authorization_service.py
session_service.py
token_service.py
service_identity.py
security_context.py
security_audit.py
```

```
security_middleware.py
```

---

93. API INTEGRATION ARTIFACTS

The OpenAPI contract shall extend with:

```
Authentication
Sessions
Current User
Project Authorization
Project Members
Roles
Capabilities
Security Events
```

Existing API resources remain unchanged unless security metadata is required.

---

94. DOMAIN INTEGRATION

The security subsystem shall integrate with:

```
Project Model Service
Aria Orchestrator
Job Service
Worker Runtime
Event Bus
Artifact Service
Compliance Engine
Validation Engine
Documentation Engine
Acceptance Service
```

No subsystem receives unrestricted authority.

---

95. SECURITY SERVICE BOUNDARY

The application architecture becomes:

```
CLIENT
  ↓
API GATEWAY
  ↓
AUTHENTICATION
  ↓
AUTHORIZATION
  ↓
RUNTIME ORCHESTRATOR
```

```
 ↓
DOMAIN SERVICES
 ↓
JOB SYSTEM
 ↓
WORKERS
 ↓
EVENT BUS
 ↓
ARTIFACTS / DATABASE
```

The security layer therefore surrounds the existing OneDraft computational architecture rather than becoming another domain model.

---

96. PRINCIPLE OF LEAST PRIVILEGE

Every actor receives the minimum authority required to perform its function.

This applies equally to:

```
users
administrators
Aria
workers
services
event consumers
scheduled processes
```

No actor receives unrestricted database authority as a convenience mechanism.

---

97. SECURITY PROVENANCE

A complete OneDraft operation should eventually be traceable as:

```
USER
 ↓
AUTHENTICATION
 ↓
AUTHORIZATION
 ↓
REQUEST
 ↓
ARIA / DOMAIN COMMAND
 ↓
REVISION
 ↓
JOB
 ↓
EVENT
 ↓
ARTIFACT
```

```
  ↓
VALIDATION
  ↓
DOCUMENT
  ↓
ACCEPTANCE
```

The security subsystem supplies the identity and authorization provenance at each protected boundary.

---

98. RECONSTRUCTION REQUIREMENT

Given:

```
request_id
user_id
organization_id
project_id
revision_id
command_id
job_id
event_id
```

the system should be able to reconstruct the security context surrounding a significant operation.

This provides the security counterpart to the existing OneDraft project reconstruction requirement.

---

99. MVP SECURITY SUCCESS CRITERIA

The security implementation is successful when it can:

Authenticate a user

Maintain a secure session

Validate an access token

Resolve organization membership

Resolve project membership

Evaluate capabilities

Authorize project operations

Authorize Aria commands

Authorize worker operations

Authorize artifact access

Authorize document release

Assign project roles

Revoke project access

Suspend accounts

Revoke credentials

Record security events

Prevent cross-project access

Prevent cross-organization access

Preserve security provenance

---

100. FIRST SECURITY INTEGRATION TEST

The first complete security integration test shall execute:

```
1. Create organization
2. Create user
3. Authenticate user
4. Establish session
5. Create project
6. Assign PROJECT_OWNER
7. Resolve project capabilities
8. Authenticate API request
9. Submit authorized project modification
10. Authorize Aria command
11. Execute model command
12. Create revision
13. Publish event
14. Execute worker operation
15. Generate artifact
16. Generate document package
17. Authorize document retrieval
18. Record acceptance
```

```
19. Retrieve complete security history
20. Revoke user project access
21. Attempt project access again
22. Verify authorization denial
23. Verify security audit event
```

This becomes the primary OneDraft security regression test.

---

101. ARCHITECTURAL INTEGRATION STATE

OneDraft now contains:

Product definition

System architecture

Technology architecture

Domain model

Persistence model

API contract

Geometry & parametric model

Compliance & regulatory engine

Validation engine

Revision system

Documentation & drawing engine

Runtime orchestrator

Job & worker execution

Artifact storage

Integration & event bus

Identity

Authentication

Authorization

Service security

Security audit

---

102. NEXT IMPLEMENTATION ARTIFACTS

The immediate implementation package is:

```
026_security_schema.sql
security_types.py
authentication_service.py
authorization_service.py
session_service.py
token_service.py
service_identity.py
security_context.py
security_audit.py
security_middleware.py
```

These artifacts establish the security boundary around the existing OneDraft runtime.

---

103. FINAL ARCHITECTURAL DECISION

OneDraft shall not treat security as a feature added after the computational system is complete.

Security is part of the computational boundary itself.

The architecture is therefore:

```
IDENTITY
+
AUTHENTICATION
+
AUTHORIZATION
+
PROJECT TENANCY
+
CAPABILITY CONTROL
+
SERVICE IDENTITY
+
AUDIT PROVENANCE
+
DOMAIN EXECUTION
```

Aria does not possess independent authority over the OneDraft system.

Aria operates through authenticated and authorized application capabilities.

Workers do not possess unrestricted authority.

Event consumers do not possess unrestricted authority.

External clients do not possess direct database authority.

The Project Model remains the authoritative computational state.

The Event Bus remains the authoritative transport mechanism for system events.

The Artifact System remains the authoritative persistence mechanism for generated artifacts.

The Security System establishes **who may cause, observe, modify, execute, release, or accept those operations**.

---

104. SPECIFICATION STATUS

ONEDRAFT IDENTITY, SECURITY & AUTHORIZATION SYSTEM v0.1

STATUS: ESTABLISHED

The security boundary is now defined as the next executable OneDraft subsystem.

The next implementation stage may therefore proceed directly into the subsequent OneDraft subsystem without redefining the underlying:

```
project
geometry
compliance
validation
revision
documentation
runtime orchestration
job execution
artifact storage
event bus
identity
security
authorization
```